# Kubernetes Pod Scheduling Controls

Node Pools · Taints & Tolerations · nodeName & nodeSelector · Topology Spread Constraints · Affinity & Anti-Affinity

**Contents**


1. Node Pools
2. Taints and Tolerations
3. nodeName and nodeSelector
4. Node Affinity & Anti-Affinity / Pod Affinity & Anti-Affinity
5. Topology Spread Constraints
6. Comparison Table & Practical Combinations


All of the mechanisms below answer the same underlying question the Kubernetes scheduler must solve for every pod: **"Which node should this pod run on?"** Each mechanism adds a different kind of control — from simple labels to expressive rules to fine-grained distribution — and in real clusters they are almost always combined.

---

## 1. Node Pools

A **node pool** (also called a *node group*) is not a native Kubernetes API object — it is a concept provided by managed Kubernetes services such as EKS, GKE, and AKS. A node pool is simply a group of nodes that share the same underlying configuration: machine type, disk type, region/zone, OS image, labels, and taints.

| Aspect | Detail |
|---|---|
| What it is | A logical group of nodes with identical specifications (instance type, disk, labels, taints) |
| Who manages it | The cloud provider's control plane / managed Kubernetes service, not the core Kubernetes API |
| Purpose | Separate different workload types onto appropriately-sized or specialized hardware |
| How pods are routed to a pool | Via labels applied to the pool's nodes, combined with `nodeSelector`, node affinity, or taints/tolerations |
| Common examples | A GPU pool for ML workloads, a spot/preemptible pool for cost savings, a high-memory pool for databases, a general-purpose pool for everything else |

> **Key idea:** a node pool by itself does nothing to route pods — it just creates a labeled/tainted group of machines. You still need `nodeSelector`/affinity to pull pods toward it, and taints to keep unwanted pods away from it.

```
Node Pool: "gpu-pool"     → nodes labeled workload=gpu,     tainted gpu=true:NoSchedule
Node Pool: "spot-pool"    → nodes labeled lifecycle=spot,   tainted spot=true:NoSchedule
Node Pool: "general-pool" → nodes labeled workload=general  (no taint)
```

## 2. Taints and Tolerations

**Taints** are applied to **nodes** and repel pods from scheduling on them. **Tolerations** are applied to **pods** and allow (but do not force) a pod to be scheduled onto a node with a matching taint. Conceptually this is a "push away unless explicitly permitted" mechanism — the opposite direction of affinity, which "pulls toward."

| Taint Effect | Behavior |
|---|---|
| `NoSchedule` | New pods without a matching toleration will not be scheduled on this node. Existing pods already running are left alone. |
| `PreferNoSchedule` | A "soft" version — the scheduler tries to avoid placing non-tolerating pods here, but it is not guaranteed. |
| `NoExecute` | New pods without a matching toleration will not be scheduled, **and** existing pods already running without the toleration are evicted. |

### Applying a taint to a node

```
kubectl taint nodes node1 gpu=true:NoSchedule
```

### Adding a matching toleration to a pod

```
tolerations:
- key: "gpu"
  operator: "Equal"
  value: "true"
  effect: "NoSchedule"
```

> **Important nuance:** a toleration only means the pod is *allowed* to be scheduled on the tainted node — it does not mean the pod will *prefer* or be *attracted to* that node. If you want the pod to actually land there (and not on some other untainted node), you must also add node affinity or a `nodeSelector` targeting that node pool's labels.

Common built-in taints include those Kubernetes automatically applies to nodes that are unreachable (`node.kubernetes.io/unreachable`) or under memory pressure — pods can tolerate these for a defined period using `tolerationSeconds`, controlling how long they stay before eviction.

## 3. nodeName and nodeSelector

These are the two simplest, most direct scheduling mechanisms Kubernetes provides.

| Mechanism | Description | Flexibility |
|---|---|---|
| `nodeName` | Directly specifies the exact node the pod must run on. This completely bypasses the scheduler — the kubelet on that node simply picks up the pod. | None — hardcoded and fragile; if the node doesn't exist or lacks capacity, the pod fails |
| `nodeSelector` | A simple key-value label match. The pod will only be scheduled on nodes that carry every label specified. | Basic — supports only exact equality matches, no OR/NOT/set-based logic |

### nodeName example

```
apiVersion: v1
kind: Pod
metadata:
  name: pinned-pod
spec:
  nodeName: node1
  containers:
  - name: app
    image: myapp:latest
```

### nodeSelector example

```
apiVersion: v1
kind: Pod
metadata:
  name: ssd-pod
spec:
  nodeSelector:
    disktype: ssd
  containers:
  - name: app
    image: myapp:latest
```

> **When to use:** `nodeName` is mainly useful for debugging or very specific edge cases (rarely recommended in production, since it removes all scheduler flexibility). `nodeSelector` is fine for simple, single-condition targeting, but for anything more expressive (multiple conditions, OR logic, soft preferences) you need node affinity instead.

## 4. Node Affinity & Anti-Affinity, Pod Affinity & Anti-Affinity

Affinity rules are a much more expressive evolution of `nodeSelector`. They support complex matching logic (`In`, `NotIn`, `Exists`, `DoesNotExist`, `Gt`, `Lt`) and can be declared as either **hard requirements** or **soft preferences**.

| Type | Targets | Purpose |
|---|---|---|
| **Node Affinity** | Labels on nodes | Attract a pod *to* nodes that match certain label criteria |
| **Node Anti-Affinity** | Labels on nodes (via `NotIn` / `DoesNotExist`) | Repel a pod *from* nodes matching certain criteria — there is no separate "node anti-affinity" API field; it is expressed using negative operators within node affinity |
| **Pod Affinity** | Labels on other running pods | Co-locate a pod near other pods (e.g., same node, same zone) — useful for reducing network latency between dependent services |
| **Pod Anti-Affinity** | Labels on other running pods | Spread a pod away from other pods matching a label — the standard way to achieve high availability across replicas |

### Rule strength

| Rule Type | Meaning |
|---|---|
| `requiredDuringSchedulingIgnoredDuringExecution` | A **hard** rule. The pod will not be scheduled at all unless the condition is satisfied. |
| `preferredDuringSchedulingIgnoredDuringExecution` | A **soft** rule. The scheduler will try to satisfy it (with a configurable weight, 1–100), but will still schedule the pod elsewhere if it can't. |

The phrase "IgnoredDuringExecution" in both names means these rules are only checked at scheduling time — if labels on a node or nearby pods change afterward, Kubernetes will not evict or reschedule already-running pods to re-satisfy the rule.

### Combined example

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
      - matchExpressions:
        - key: disktype
          operator: In
          values: ["ssd"]
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector:
        matchExpressions:
        - key: app
          operator: In
          values: ["web"]
      topologyKey: "kubernetes.io/hostname"
```

This example schedules the pod only on SSD-backed nodes, and additionally guarantees that no two pods labeled `app=web` ever land on the same physical node (host) — a classic high-availability pattern for replica sets.

### Pod affinity example (co-location)

```
affinity:
  podAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector:
        matchExpressions:
        - key: app
          operator: In
          values: ["cache"]
      topologyKey: "kubernetes.io/hostname"
```

This pulls the pod onto the same node as any pod labeled `app=cache` — useful when a frontend needs very low-latency access to a local cache.

## 5. Topology Spread Constraints

Topology Spread Constraints give the scheduler a precise, quantitative way to **evenly distribute** pods across failure domains such as nodes, zones, or regions. They achieve a similar goal to pod anti-affinity (spreading replicas apart), but with much finer control over the actual balance, rather than a simple yes/no co-location rule.

| Field | Purpose |
|---|---|

| | |
|---|---|
| `topologyKey` | The failure domain to spread across — e.g. `topology.kubernetes.io/zone`, `kubernetes.io/hostname`, or a custom region label |
| `maxSkew` | The maximum allowed difference in matching-pod count between the most-loaded and least-loaded domain |
| `whenUnsatisfiable` | `DoNotSchedule` (hard — refuse to schedule if it would break the constraint) or `ScheduleAnyway` (soft — schedule anyway, but keep trying to minimize skew) |
| `labelSelector` | Identifies which pods are counted when calculating the spread (typically pods belonging to the same Deployment/app) |

```
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule
  labelSelector:
    matchLabels:
      app: web
```

This ensures `app=web` pods are spread evenly across all available zones, with at most a difference of 1 pod between the busiest and least busy zone — protecting the application from a single zone outage taking down a disproportionate share of replicas.

> **Topology Spread Constraints vs. Pod Anti-Affinity:** anti-affinity is essentially binary (pods either can or cannot co-exist in the same domain), while topology spread constraints allow a tunable, numeric balance (e.g., "at most 1 pod difference between zones") and can spread across multiple domains simultaneously with cleaner syntax. For most modern HA use cases, topology spread constraints are the preferred, more flexible tool.

## 6. Comparison Table & Practical Combinations

| Mechanism | Direction | Granularity | Hard/Soft | Common Use Case |
|---|---|---|---|---|
| **nodeName** | Pod → specific node | Exact node | Hard only | Debugging, rare edge cases |
| **nodeSelector** | Pod → node label | Simple exact match | Hard only | Basic node targeting (e.g. `disktype=ssd`) |
| **Taints / Tolerations** | Node repels pod | Node-level | Both (`NoSchedule` / `PreferNoSchedule` / `NoExecute`) | Dedicating nodes to specific workloads (GPU, spot instances) |
| **Node Affinity** | Pod → node label | Expressive (`In` / `NotIn` / `Exists`) | Both | Complex node-targeting logic |
| **Pod Affinity** | Pod → other pods | Co-locate | Both | Placing a cache next to its consuming app |
| **Pod Anti-Affinity** | Pod ← other pods | Spread apart | Both | High availability — spreading replicas across nodes/zones |
| **Topology Spread Constraints** | Even distribution | Fine-grained skew control | Both | Precisely balancing replicas evenly across zones/nodes |

### How these combine in real clusters

| Goal | Typical Combination |
|---|---|
| Dedicate a node pool to a specific workload | Taint the pool's nodes + add a matching toleration on the pod + add node affinity/selector so the pod is actually *attracted* there (a taint alone only permits, it does not pull) |
| High availability across zones | Topology spread constraints, or pod anti-affinity with `topologyKey: zone` |
| Keep a frontend near its local cache | Pod affinity with `topologyKey: kubernetes.io/hostname` |
| Avoid noisy neighbors on GPU nodes | Taint GPU nodes with `NoSchedule`; only GPU-requiring pods carry the matching toleration |

> **Summary:** `nodeName` / `nodeSelector` answer "put it here" in the simplest way; taints/tolerations answer "who is even allowed here"; affinity/anti-affinity answer "what should it be near or far from, and how strictly"; and topology spread constraints answer "how evenly should replicas be balanced across failure domains." Production clusters typically layer several of these together rather than relying on just one.